

■ Java Programming

Arrays & Strings

A Complete Beginner-to-Advanced Learning Guide

■ Beginner Friendly

■ Code-First

■ Interview Ready

■ **Topics Covered:** Arrays • Strings • StringBuilder • Interview Problems

■ **Audience:** Beginners → Intermediate Java Students

■ **Includes:** 50+ Code Examples • Exercises • Real-World Use Cases

■ **Sections:** 7 Major Sections covering all core concepts

Table of Contents

1. Introduction

- What are Arrays and Strings?
- Why are they important?

2. Arrays in Java

- Definition & Types (1D, 2D, Multidimensional)
- Declaration, Initialization & Access
- Iterating with Loops
- Array Utilities & Common Operations
- Searching, Sorting & Copying
- Passing/Returning Arrays from Methods
- Common Mistakes & Best Practices

3. Strings in Java

- Definition & String Creation Methods
- String vs StringBuilder vs StringBuffer
- String Immutability
- Important String Methods (11 methods covered)
- String Comparison, Manipulation & Formatting
- Common Mistakes & Best Practices

4. Interview Practice Problems

- 15 Coding Problems with Solutions
- Reverse, Palindrome, Duplicates, Sorting & more

5. Mini Exercises for Students

- 10 Beginner Exercises
- 5 Intermediate Challenges

6. Real-World Examples

- Arrays & Strings in actual applications

7. Conclusion

- Summary of key concepts & next steps

Section 1: Introduction

Understanding Arrays and Strings in Java

1.1 What are Arrays in Java?

An **array** is a container object in Java that holds a fixed number of values of a single type. Think of it like a row of numbered mailboxes in an apartment building — each box holds one item, and you can quickly reach any box by its number (index).

- All elements must be of the **same data type** (e.g., all integers, all Strings).
- The **size is fixed** at the time of creation and cannot be changed.
- Arrays are **zero-indexed**: the first element is at index 0.
- Arrays are **objects** in Java, stored on the heap.

```
// A simple integer array holding 5 elements
int[] scores = {95, 87, 76, 91, 83};

// Accessing elements (zero-indexed)
System.out.println(scores[0]); // Output: 95
System.out.println(scores[4]); // Output: 83
```

1.2 What are Strings in Java?

A **String** in Java is an object that represents a sequence of characters. Unlike arrays, Strings have a rich built-in API with dozens of helper methods, making them indispensable for text-based processing.

- Strings are instances of the **java.lang.String** class.
- They are **immutable** — once created, their content cannot change.
- Java provides the **String pool** (intern pool) for memory efficiency.
- Strings can be created using **literals** or the **new** keyword.

```
// String literal (stored in String pool)
String greeting = "Hello, World!";

// Using new keyword (stored on heap)
String name = new String("Alice");

// Printing string length
System.out.println(greeting.length()); // Output: 13
```

1.3 Why Are They Important in Programming?

Arrays and Strings are among the **most frequently used data structures** in software development. Every real-world application — from web servers to mobile apps — relies on them heavily.

Use Case	Arrays Used For	Strings Used For
----------	-----------------	------------------

Web Applications	Store form field values	Parse HTTP requests/URLs
Data Analytics	Store numerical datasets	Read CSV column headers
Games	Store player scores	Display player names/messages
Banking Systems	Hold transaction amounts	Format account numbers
Search Engines	Store ranked results	Tokenize search queries

■ Quick Tip

- Mastering Arrays and Strings is essential for coding interviews.
- Most beginner-to-intermediate interview problems involve one or both.
- Understanding their internals helps you write faster, cleaner code.

Section 2: Arrays in Java

From declaration to advanced operations

2.1 Definition of Arrays

An array is a **fixed-size, ordered collection** of elements of the same data type, stored in contiguous memory locations. In Java, arrays are objects and are always allocated on the heap.

Property	Description
Type	Homogeneous — all elements share the same type
Size	Fixed at creation; cannot grow or shrink
Index	Zero-based (first element = index 0)
Storage	Elements stored in contiguous memory
Access	O(1) constant-time random access via index

2.2 Types of Arrays

2.2.1 One-Dimensional Array

A simple list of elements arranged in a single row.

```
// Declaration and initialization
int[] numbers = {10, 20, 30, 40, 50};

// Declaration then initialization
String[] fruits = new String[3];
fruits[0] = "Apple";
fruits[1] = "Banana";
fruits[2] = "Cherry";

// Printing all elements
for (int i = 0; i < numbers.length; i++) {
    System.out.println("Element at " + i + ": " + numbers[i]);
}

// Output:
// Element at 0: 10
// Element at 1: 20 ... and so on
```

2.2.2 Two-Dimensional Array

A 2D array is an array of arrays — like a table (matrix) with rows and columns.

```
// 3x3 matrix
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Access element at row 1, column 2
System.out.println(matrix[1][2]); // Output: 6

// Print all elements of 2D array
for (int row = 0; row < matrix.length; row++) {
    for (int col = 0; col < matrix[row].length; col++) {
        System.out.print(matrix[row][col] + " ");
    }
    System.out.println();
}
// Output:
// 1 2 3
// 4 5 6
// 7 8 9
```

2.2.3 Multidimensional Arrays

Java supports arrays with more than two dimensions, though 3D arrays are the most common beyond 2D. These are used in scientific computing, 3D game development, and similar domains.

```
// 3D array: 2 layers, 2 rows, 3 columns
int[][][] cube = {
    {{1,2,3}, {4,5,6}},
    {{7,8,9}, {10,11,12}}
};

// Access element: layer 1, row 0, column 2
System.out.println(cube[1][0][2]); // Output: 9
```

2.3 Array Declaration and Initialization

Style	Example	Notes
Declare only	<code>int[] arr;</code>	No memory allocated yet
Declare + allocate	<code>int[] arr = new int[5];</code>	Default values (0 for int)
Declare + init	<code>int[] arr = {1,2,3};</code>	Size inferred from values
Anonymous array	<code>new int[]{1,2,3}</code>	Useful in method arguments

Default values when using `new int[n]`:

- **int, byte, short, long** → 0
- **float, double** → 0.0

- **boolean** → false
- **char** → '\u0000' (null character)
- **Objects / Strings** → null

2.4 Accessing Array Elements

Elements are accessed using their **index** (position). Java uses **zero-based indexing**, so the first element is at index 0 and the last is at index *length - 1*.

```
int[] data = {100, 200, 300, 400, 500};

System.out.println(data[0]);           // 100 (first)
System.out.println(data[4]);           // 500 (last)
System.out.println(data[data.length-1]); // 500 (safer last element)

// Modifying an element
data[2] = 999;
System.out.println(data[2]);           // 999
```

■ ■ **ArrayIndexOutOfBoundsException**

- Accessing index -1 or $\geq$ array.length causes a runtime exception.
- Always validate: if (index $\geq$ 0 && index < arr.length) before access.
- This is the most common beginner mistake with arrays!

2.5 Iterating Arrays Using Loops

for Loop — when you need the index

```
int[] marks = {85, 92, 78, 95, 88};

for (int i = 0; i < marks.length; i++) {
    System.out.println("Student " + (i+1) + ": " + marks[i]);
}
```

while Loop — useful with external conditions

```
int[] marks = {85, 92, 78, 95, 88};
int i = 0;
while (i < marks.length) {
    System.out.println(marks[i]);
    i++;
}
```

Enhanced for Loop (for-each) — clean and readable

```
int[] marks = {85, 92, 78, 95, 88};

for (int mark : marks) {
    System.out.println("Mark: " + mark);
}

// Note: You CANNOT modify the array using enhanced for-loop
// Use the regular for loop if you need to update values.
```

Loop Type	Can Modify?	Has Index?	Best Used When
for	Yes	Yes	Need index or modifying elements
while	Yes	Yes	Loop count known by condition
Enhanced for	No	No	Read-only iteration over all elements

2.6 Array Methods and Utilities (java.util.Arrays)

The **java.util.Arrays** class provides many powerful static methods that save you from writing manual loops for common tasks.

```
import java.util.Arrays;

int[] nums = {5, 2, 8, 1, 9, 3};

// Sort the array
Arrays.sort(nums);
System.out.println(Arrays.toString(nums)); // [1, 2, 3, 5, 8, 9]

// Binary search (array must be sorted first!)
int index = Arrays.binarySearch(nums, 8);
System.out.println(index); // 4

// Fill array with a value
int[] zeros = new int[5];
Arrays.fill(zeros, 7);
System.out.println(Arrays.toString(zeros)); // [7, 7, 7, 7, 7]

// Copy array
int[] copy = Arrays.copyOf(nums, 3);
System.out.println(Arrays.toString(copy)); // [1, 2, 3]

// Check equality
int[] a = {1,2,3};
int[] b = {1,2,3};
System.out.println(Arrays.equals(a, b)); // true
```

Method	Description
<code>Arrays.sort(arr)</code>	Sorts array in ascending order

<code>Arrays.toString(arr)</code>	Returns printable string of array
<code>Arrays.binarySearch(arr, key)</code>	Finds index of key (sorted array only)
<code>Arrays.fill(arr, val)</code>	Fills all elements with val
<code>Arrays.copyOf(arr, len)</code>	Copies first len elements to new array
<code>Arrays.copyOfRange(arr,f,t)</code>	Copies elements from index f to t-1
<code>Arrays.equals(a, b)</code>	Returns true if arrays are equal
<code>Arrays.deepToString(arr)</code>	For printing 2D/multi-dim arrays

2.7 Common Array Operations

Searching — Linear Search

```
public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i; // Return the index
        }
    }
    return -1; // Not found
}

// Usage
int[] data = {10, 25, 47, 3, 91};
System.out.println(linearSearch(data, 47)); // Output: 2
System.out.println(linearSearch(data, 50)); // Output: -1
```

Sorting — Bubble Sort (manual implementation)

```
public static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j+1]) {
                // Swap arr[j] and arr[j+1]
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

int[] nums = {64, 25, 12, 22, 11};
bubbleSort(nums);
System.out.println(Arrays.toString(nums)); // [11, 12, 22, 25, 64]
```

Copying Arrays

```
int[] original = {1, 2, 3, 4, 5};

// Method 1: Arrays.copyOf()
int[] copy1 = Arrays.copyOf(original, original.length);

// Method 2: System.arraycopy()
int[] copy2 = new int[original.length];
System.arraycopy(original, 0, copy2, 0, original.length);

// Method 3: Manual loop
int[] copy3 = new int[original.length];
for (int i = 0; i < original.length; i++) {
    copy3[i] = original[i];
}

// Method 4: clone()
int[] copy4 = original.clone();
```

■ Shallow vs Deep Copy

- For primitive arrays, all copy methods work the same — they copy values.
- For Object arrays, copies only copy references (shallow copy).
- Modifying an object in a shallow copy affects the original too.
- Use deep copy logic for arrays of mutable objects.

2.8 Passing Arrays to Methods

In Java, arrays are passed **by reference**. This means the method receives a reference to the original array — changes inside the method *will* affect the original array.

```
public static void doubleValues(int[] arr) {
    for (int i = 0; i < arr.length; i++) {
        arr[i] *= 2; // Modifies the ORIGINAL array
    }
}

public static void main(String[] args) {
    int[] numbers = {1, 2, 3, 4, 5};
    doubleValues(numbers);
    System.out.println(Arrays.toString(numbers));
    // Output: [2, 4, 6, 8, 10]
}
```

2.9 Returning Arrays from Methods

```
public static int[] generateSquares(int n) {
    int[] squares = new int[n];
    for (int i = 0; i < n; i++) {
        squares[i] = (i + 1) * (i + 1);
    }
    return squares;
}

public static void main(String[] args) {
    int[] result = generateSquares(5);
    System.out.println(Arrays.toString(result));
    // Output: [1, 4, 9, 16, 25]
}
```

2.10 Practical Example: Student Grade Calculator

```
import java.util.Arrays;

public class GradeCalculator {

    public static double average(int[] grades) {
        int sum = 0;
        for (int g : grades) sum += g;
        return (double) sum / grades.length;
    }

    public static int highest(int[] grades) {
        int max = grades[0];
        for (int g : grades) if (g > max) max = g;
        return max;
    }

    public static int lowest(int[] grades) {
        int min = grades[0];
        for (int g : grades) if (g < min) min = g;
        return min;
    }

    public static void main(String[] args) {
        int[] grades = {85, 92, 78, 95, 88, 74, 91};
        System.out.println("Average : " + average(grades)); // 86.14
        System.out.println("Highest : " + highest(grades)); // 95
        System.out.println("Lowest : " + lowest(grades)); // 74

        Arrays.sort(grades);
        System.out.println("Sorted : " + Arrays.toString(grades));
        // [74, 78, 85, 88, 91, 92, 95]
    }
}
```

2.11 Common Mistakes Beginners Make with Arrays

■ ■ Watch Out For These!

- Off-by-one errors: using `<=` instead of `<` in loop conditions.
- Forgetting arrays are zero-indexed: `arr[1]` is the 2nd element.
- Not initializing the array before using it (`NullPointerException`).
- Comparing arrays with `==` instead of `Arrays.equals()`.
- Assuming array assignment (`b=a`) creates a copy — it only copies the reference.
- Accessing elements beyond the array length (`ArrayIndexOutOfBoundsException`).

2.12 Best Practices for Arrays

■ Array Best Practices

- Always check array length before accessing elements.
- Use `Arrays.toString()` for printing — never print the array variable directly.
- Prefer `ArrayList` when you need a dynamic-size collection.
- Use `Arrays.sort()` instead of writing your own sorting algorithms in production.
- Name arrays clearly: `studentGrades` not `sg` or `data`.
- Use `final` for arrays that should not be reassigned.
- For 2D arrays, store dimensions in variables to avoid repeated `.length` calls.

Section 3: Strings in Java

Mastering Java's most-used class

3.1 Definition of String

A **String** in Java is an object of the **java.lang.String** class that represents an immutable sequence of Unicode characters. The String class implements *Serializable*, *Comparable*, and *CharSequence*.

- Strings are objects, not primitives (unlike int or char).
- Every String literal is automatically interned in the **String pool**.
- Strings are **immutable** — any 'modification' creates a new object.
- The String class has **60+ built-in methods** for text manipulation.

3.2 String Creation Methods

```
// Method 1: String literal (recommended – uses String pool)
String s1 = "Hello";

// Method 2: new keyword (creates a new object every time)
String s2 = new String("Hello");

// Method 3: From char array
char[] chars = {'J', 'a', 'v', 'a'};
String s3 = new String(chars); // "Java"

// Method 4: From StringBuilder
StringBuilder sb = new StringBuilder("World");
String s4 = sb.toString(); // "World"

// Method 5: valueOf
String s5 = String.valueOf(42); // "42"
String s6 = String.valueOf(3.14); // "3.14"
String s7 = String.valueOf(true); // "true"
```

3.3 String vs StringBuilder vs StringBuffer

Java provides three classes for working with text. Choosing the right one affects both correctness and performance.

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread Safe	Yes	No	Yes
Performance	Slow (concat)	Fast	Moderate
Memory	High (GC)	Efficient	Efficient

Use When	Few operations	Single thread	Multi-thread
----------	----------------	---------------	--------------

```
// String concatenation (creates new object each time – SLOW in loops)
String result = "";
for (int i = 0; i < 5; i++) {
    result += i; // 5 new String objects created!
}

// StringBuilder (EFFICIENT – modifies in-place)
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++) {
    sb.append(i);
}
String result2 = sb.toString(); // "01234"

// Useful StringBuilder methods
sb.append(" World");
sb.insert(0, 'H');
sb.reverse();
sb.delete(0, 2);
sb.replace(0, 2, "Hi");
```

3.4 String Immutability

String immutability means that once a String object is created, its contents **cannot be changed**. Any operation that appears to modify a String actually creates a **new String object**.

```
String original = "Hello";

// This does NOT modify original – it creates a new String
String modified = original.concat(" World");

System.out.println(original); // "Hello" (unchanged)
System.out.println(modified); // "Hello World" (new object)

// Why immutability matters:
// 1. Thread safety – safe to share across threads
// 2. Caching – hash code computed once and cached
// 3. Security – passwords/URLs can't be altered accidentally
```

3.5 Important String Methods

• `length()`

Returns the number of characters in the String.

```
String s = "Hello Java";
System.out.println(s.length()); // 10
```

• `charAt(int index)`

Returns the character at the specified index.

```
String s = "Hello";
System.out.println(s.charAt(0)); // H
System.out.println(s.charAt(4)); // o
```

- **substring(int start, int end)**

Extracts part of the string from start to end-1.

```
String s = "Hello World";
System.out.println(s.substring(6)); // "World"
System.out.println(s.substring(0, 5)); // "Hello"
```

- **equals(Object obj)**

Compares string content (case-sensitive).

```
String a = "Java";
String b = "Java";
System.out.println(a.equals(b)); // true
System.out.println(a.equals("java")); // false
```

- **equalsIgnoreCase(String)**

Compares string content ignoring case.

```
String a = "Java";
System.out.println(a.equalsIgnoreCase("java")); // true
System.out.println(a.equalsIgnoreCase("JAVA")); // true
```

- **toUpperCase()**

Converts all characters to uppercase.

```
String s = "hello world";
System.out.println(s.toUpperCase()); // HELLO WORLD
```

- **toLowerCase()**

Converts all characters to lowercase.

```
String s = "HELLO WORLD";
System.out.println(s.toLowerCase()); // hello world
```

- **trim()**

Removes leading and trailing whitespace.

```
String s = "  Hello  ";
System.out.println(s.trim());           // "Hello"
System.out.println(s.trim().length()); // 5
```

• **replace(old, new)**

Replaces all occurrences of a character or substring.

```
String s = "banana";
System.out.println(s.replace("a", "o")); // bonono
System.out.println(s.replace('a', '*')); // b*n*n*
```

• **contains(CharSequence)**

Returns true if the string contains the given sequence.

```
String s = "Hello World";
System.out.println(s.contains("World")); // true
System.out.println(s.contains("world")); // false (case-sensitive)
```

• **split(String regex)**

Splits the string into an array using a delimiter.

```
String csv = "Alice,Bob,Charlie,Dave";
String[] names = csv.split(",");
for (String name : names) {
    System.out.println(name);
}
// Alice
// Bob
// Charlie
// Dave
```

3.6 String Comparison

String comparison is one of the most misunderstood topics for beginners. There are two ways to compare Strings, and they mean very different things.


```
String s1 = new String("Java");
String s2 = new String("Java");

// == compares REFERENCES (memory addresses)
System.out.println(s1 == s2);           // false (different objects)

// .equals() compares CONTENT
System.out.println(s1.equals(s2));      // true

// String pool example
String s3 = "Java";
String s4 = "Java";
System.out.println(s3 == s4);           // true (same pool reference)
System.out.println(s3.equals(s4));      // true

// Comparing order: compareTo()
System.out.println("Apple".compareTo("Banana")); // negative (Apple < Banana)
System.out.println("Zebra".compareTo("Apple"));  // positive (Zebra > Apple)
System.out.println("Java".compareTo("Java"));    // 0 (equal)
```

■■ Never use == to compare Strings!

- Always use `.equals()` for content comparison.
- Use `.equalsIgnoreCase()` when case doesn't matter.
- Use `.compareTo()` when you need lexicographic ordering.

3.7 String Manipulation Techniques

```
// Reversing a String
String original = "Hello";
String reversed = new StringBuilder(original).reverse().toString();
System.out.println(reversed); // "olleH"

// Checking palindrome
String word = "racecar";
boolean isPalindrome = word.equals(new StringBuilder(word).reverse().toString());
System.out.println(isPalindrome); // true

// Counting occurrences of a character
String sentence = "hello world";
long count = sentence.chars().filter(c -> c == 'l').count();
System.out.println(count); // 3

// Joining strings
String joined = String.join(", ", "Alice", "Bob", "Charlie");
System.out.println(joined); // "Alice, Bob, Charlie"

// Repeating a string (Java 11+)
String repeated = "Java! ".repeat(3);
System.out.println(repeated); // "Java! Java! Java! "

// Checking blank/empty
System.out.println("").isEmpty(); // true
System.out.println(" ".isBlank()); // true (Java 11+)
```

3.8 String Formatting

```
// Method 1: String.format()
String name = "Alice";
int age = 25;
double gpa = 3.95;
String info = String.format("Name: %s | Age: %d | GPA: %.2f", name, age, gpa);
System.out.println(info);
// Name: Alice | Age: 25 | GPA: 3.95

// Method 2: printf()
System.out.printf("%-15s %5d %8.2f%n", name, age, gpa);
// Alice          25      3.95

// Method 3: Formatted (Java 15+)
String s = "Hello %s!".formatted(name);
System.out.println(s); // Hello Alice!

// Format specifiers quick reference
// %s = String    %d = integer    %f = float
// %c = char      %b = boolean    %n = newline
// %.2f = 2 decimal places
// %5d = right-pad to width 5
// %-10s = left-align in width 10
```

3.9 Practical Example: Word Frequency Counter

```
import java.util.HashMap;
import java.util.Map;

public class WordFrequency {
    public static void main(String[] args) {
        String text = "the cat sat on the mat the cat";

        // Split into words
        String[] words = text.split(" ");

        // Count frequency
        Map<String, Integer> freq = new HashMap<>();
        for (String word : words) {
            freq.put(word, freq.getOrDefault(word, 0) + 1);
        }

        // Print results
        for (Map.Entry<String, Integer> entry : freq.entrySet()) {
            System.out.println(entry.getKey() + " -> " + entry.getValue());
        }
        // the -> 3
        // cat -> 2
        // sat -> 1
        // on -> 1
        // mat -> 1
    }
}
```

3.10 Common Beginner Mistakes with Strings

■■ String Pitfalls to Avoid

- Using `==` instead of `.equals()` for comparison.
- String concatenation inside loops — use `StringBuilder` instead.
- Not handling null before calling `.equals()` — causes `NullPointerException`.
- Ignoring the return value of string methods (Strings are immutable!).
- Using `.replace()` thinking it modifies the original — it doesn't.
- Not trimming user input before comparison or storage.
- Confusing `charAt(0)` with `substring(0,1)`: one returns `char`, the other `String`.

3.11 Best Practices for Strings

■ String Best Practices

- Use `"literal".equals(variable)` to avoid `NullPointerException`.
- Use `StringBuilder` for string building in loops.
- Always `trim()` user input from forms or file reading.
- Prefer `isEmpty()` and `isBlank()` over `length() == 0` checks.
- Use `String.join()` for clean list-to-string conversion.
- Intern frequently reused strings to save memory.
- Use switch statements with `String` (Java 7+) for clean branching.

Section 4: Interview Practice Problems

15 Classic Coding Problems with Java Solutions

These problems are commonly asked in technical interviews for entry and junior-level Java positions. Study each solution carefully and practice writing them from memory.

Problem 1: Reverse a String

Write a method that reverses a given string without using `StringBuilder.reverse()`.

Two-pointer technique — $O(n)$ time, $O(n)$ space

```
public static String reverse(String s) {
    char[] chars = s.toCharArray();
    int left = 0, right = chars.length - 1;
    while (left < right) {
        char temp = chars[left];
        chars[left] = chars[right];
        chars[right] = temp;
        left++; right--;
    }
    return new String(chars);
}
// reverse("hello") -> "olleh"
```

Problem 2: Check Palindrome

Determine if a given string reads the same forwards and backwards.

Two-pointer after cleaning the string

```
public static boolean isPalindrome(String s) {
    s = s.toLowerCase().replaceAll("[^a-z0-9]", "");
    int l = 0, r = s.length() - 1;
    while (l < r) {
        if (s.charAt(l) != s.charAt(r)) return false;
        l++; r--;
    }
    return true;
}
// isPalindrome("racecar") -> true
// isPalindrome("hello") -> false
```

Problem 3: Find Duplicate Characters

Print all characters that appear more than once in a string.

Frequency array approach — $O(n)$ time

```
public static void findDuplicates(String s) {
    int[] freq = new int[256]; // ASCII
    for (char c : s.toCharArray()) freq[c]++;
    System.out.print("Duplicates: ");
    for (int i = 0; i < 256; i++) {
        if (freq[i] > 1) System.out.print((char)i + " ");
    }
}
// findDuplicates("programming") -> g r m
```

Problem 4: Sum of Array Elements

Calculate and return the sum of all elements in an integer array.

```
public static int arraySum(int[] arr) {
    int sum = 0;
    for (int x : arr) sum += x;
    return sum;
}
// arraySum(new int[]{1,2,3,4,5}) -> 15
```

Problem 5: Largest Number in Array

Find and return the largest element in an integer array.

```
public static int findMax(int[] arr) {
    int max = arr[0];
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] > max) max = arr[i];
    }
    return max;
}
// findMax(new int[]{3, 7, 1, 9, 2}) -> 9
```

Problem 6: Frequency of Each Character

Count how many times each character appears in a string.

```
public static void charFrequency(String s) {
    Map<Character, Integer> map = new LinkedHashMap<>();
    for (char c : s.toCharArray()) {
        map.put(c, map.getOrDefault(c, 0) + 1);
    }
    for (Map.Entry<Character, Integer> e : map.entrySet()) {
        System.out.println("'" + e.getKey() + "' -> " + e.getValue());
    }
}
// charFrequency("banana"):
// 'b' -> 1 'a' -> 3 'n' -> 2
```

Problem 7: Remove Spaces from a String

Remove all whitespace characters from a string.

```
// Method 1: replaceAll
public static String removeSpaces(String s) {
    return s.replaceAll("\\s+", "");
}

// Method 2: StringBuilder
public static String removeSpaces2(String s) {
    StringBuilder sb = new StringBuilder();
    for (char c : s.toCharArray()) {
        if (c != ' ') sb.append(c);
    }
    return sb.toString();
}
// removeSpaces("Hello World") -> "HelloWorld"
```

Problem 8: Second Largest Element in Array

Find the second largest unique element in an integer array.

Single pass $O(n)$ solution

```
public static int secondLargest(int[] arr) {
    int first = Integer.MIN_VALUE;
    int second = Integer.MIN_VALUE;
    for (int x : arr) {
        if (x > first) { second = first; first = x; }
        else if (x > second && x != first) second = x;
    }
    return second;
}
// secondLargest({3,1,9,5,7}) -> 7
```

Problem 9: Count Vowels and Consonants

Count the number of vowels and consonants in a string.

```
public static void countVowelsConsonants(String s) {
    int vowels = 0, consonants = 0;
    s = s.toLowerCase();
    for (char c : s.toCharArray()) {
        if ("aeiou".indexOf(c) != -1) vowels++;
        else if (c >= 'a' && c <= 'z') consonants++;
    }
    System.out.println("Vowels: " + vowels);
    System.out.println("Consonants: " + consonants);
}
// countVowelsConsonants("Hello World")
// Vowels: 3  Consonants: 7
```

Problem 10: Check Anagram

Determine if two strings are anagrams of each other.

O(n) using frequency array

```
public static boolean isAnagram(String s1, String s2) {
    if (s1.length() != s2.length()) return false;
    int[] freq = new int[256];
    for (char c : s1.toCharArray()) freq[c]++;
    for (char c : s2.toCharArray()) freq[c]--;
    for (int f : freq) if (f != 0) return false;
    return true;
}
// isAnagram("listen", "silent") -> true
// isAnagram("hello", "world") -> false
```

Problem 11: Rotate Array

Rotate an integer array to the right by k positions.

Three-reverse trick — O(n) in-place

```
public static void rotate(int[] arr, int k) {
    int n = arr.length;
    k = k % n;
    reverse(arr, 0, n-1);
    reverse(arr, 0, k-1);
    reverse(arr, k, n-1);
}
private static void reverse(int[] arr, int l, int r) {
    while (l < r) {
        int tmp = arr[l]; arr[l] = arr[r]; arr[r] = tmp;
        l++; r--;
    }
}
// rotate({1,2,3,4,5}, 2) -> {4,5,1,2,3}
```


Problem 12: Find First Non-Repeating Character

Return the first character in a string that does not repeat.

```
public static char firstUnique(String s) {
    int[] freq = new int[256];
    for (char c : s.toCharArray()) freq[c]++;
    for (char c : s.toCharArray()) {
        if (freq[c] == 1) return c;
    }
    return '\0'; // no unique character found
}
// firstUnique("aabbccdef") -> 'c'
```

Problem 13: Convert String to Integer (atoi)

Implement a basic version of `parseInt` without using `Integer.parseInt()`.

```
public static int myAtoi(String s) {
    s = s.trim();
    int sign = 1, result = 0, i = 0;
    if (i < s.length() && (s.charAt(i) == '-' || s.charAt(i) == '+')) {
        sign = (s.charAt(i++) == '-') ? -1 : 1;
    }
    while (i < s.length() && Character.isDigit(s.charAt(i))) {
        result = result * 10 + (s.charAt(i++) - '0');
    }
    return sign * result;
}
// myAtoi(" -42") -> -42
// myAtoi("123abc") -> 123
```

Problem 14: Remove Duplicates from Array

Remove all duplicate values from a sorted array in-place.

Two-pointer approach — $O(n)$ in-place

```
public static int removeDuplicates(int[] arr) {
    if (arr.length == 0) return 0;
    int k = 1; // next write position
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] != arr[k-1]) {
            arr[k++] = arr[i];
        }
    }
    return k; // new length
}
// {1,1,2,3,3,4} -> {1,2,3,4}, returns 4
```

Problem 15: Longest Common Prefix

Find the longest common prefix string among an array of strings.

```
public static String longestPrefix(String[] strs) {
    if (strs.length == 0) return "";
    String prefix = strs[0];
    for (int i = 1; i < strs.length; i++) {
        while (!strs[i].startsWith(prefix)) {
            prefix = prefix.substring(0, prefix.length()-1);
            if (prefix.isEmpty()) return "";
        }
    }
    return prefix;
}
// {"flower", "flow", "flight"} -> "fl"
// {"dog", "racecar", "car"} -> ""
```

Section 5: Mini Exercises for Students

Practice makes perfect — try these on your own!

5.1 Beginner Exercises (Arrays)

1. Print all elements of an array using a for loop.
2. Calculate the sum of all elements in an integer array.
3. Find the minimum value in an array without using `Arrays.sort()`.
4. Count how many even numbers exist in an array.
5. Swap the first and last elements of an array.
6. Copy all elements of one array into another array manually.
7. Print the array in reverse order without reversing the array itself.
8. Find how many elements in an array are greater than the average.
9. Create an array of 10 numbers and print only the ones divisible by 3.
10. Merge two arrays into a single larger array.

5.2 Beginner Exercises (Strings)

1. Count the number of vowels in the string "Hello World".
2. Convert a user-entered string to title case (first letter of each word capitalized).
3. Check if a string starts with a capital letter.
4. Replace every space in a string with an underscore.
5. Print each character of a string on a separate line using a for-each loop.

5.3 Intermediate Exercises

Challenge 1: Array Challenge

Given an integer array, move all zeros to the end while maintaining the relative order of non-zero elements. Do it in-place without extra space.

Challenge 2: String Builder

Write a method that takes a sentence and returns a new sentence where each word is reversed individually but the word order stays the same. Example: 'Hello World' → 'olleH dlroW'

Challenge 3: 2D Array Diagonal

For a 3×3 matrix, print the sum of both diagonals (main diagonal and anti-diagonal) separately.

Challenge 4: String Compression

Implement basic string compression: 'aabcccd' → 'a2b1c3d'. If the compressed string is not smaller, return the original.

Challenge 5: Array Intersection

Given two integer arrays, find all elements that appear in both arrays (intersection). Return the result without duplicates.

Section 6: Real-World Examples

How Arrays and Strings power real applications

6.1 E-Commerce: Shopping Cart

Online stores use arrays to manage product lists, prices, and cart items. Strings handle product names, descriptions, and search queries.

```
public class ShoppingCart {
    private String[] items = new String[100];
    private double[] prices = new double[100];
    private int count = 0;

    public void addItem(String name, double price) {
        items[count] = name;
        prices[count] = price;
        count++;
    }

    public double totalCost() {
        double total = 0;
        for (int i = 0; i < count; i++) total += prices[i];
        return total;
    }

    public void printReceipt() {
        System.out.println("=== RECEIPT ===");
        for (int i = 0; i < count; i++) {
            System.out.printf("%-20s $%.2f\n", items[i], prices[i]);
        }
        System.out.printf("%-20s $%.2f\n", "TOTAL", totalCost());
    }
}
```

6.2 Data Processing: CSV File Parser

Reading CSV files is a common data-processing task. Strings and `split()` make CSV parsing straightforward in Java.

```
public class CSVParser {
    public static void parseCSVLine(String line) {
        String[] fields = line.split(",");
        System.out.println("Name : " + fields[0].trim());
        System.out.println("Age : " + fields[1].trim());
        System.out.println("City : " + fields[2].trim());
    }

    public static void main(String[] args) {
        String csvLine = "Alice, 28, New York";
        parseCSVLine(csvLine);
        // Name : Alice
        // Age : 28
        // City : New York
    }
}
```

6.3 Security: Password Validator

Security systems use String methods to enforce password policies — checking length, character types, and patterns.

```
public class PasswordValidator {
    public static boolean isStrong(String pwd) {
        if (pwd.length() < 8) return false;

        boolean hasUpper = false, hasLower = false;
        boolean hasDigit = false, hasSpecial = false;

        for (char c : pwd.toCharArray()) {
            if (Character.isUpperCase(c)) hasUpper = true;
            if (Character.isLowerCase(c)) hasLower = true;
            if (Character.isDigit(c)) hasDigit = true;
            if ("!@#$%^&*".indexOf(c) >= 0) hasSpecial = true;
        }

        return hasUpper && hasLower && hasDigit && hasSpecial;
    }

    public static void main(String[] args) {
        System.out.println(isStrong("Abc@1234")); // true
        System.out.println(isStrong("password")); // false
    }
}
```

6.4 Image Processing: Pixel Grid (2D Array)

Images in memory are represented as 2D arrays where each cell stores pixel color data. This example shows grayscale conversion.

```
// Represents a simple 3x3 grayscale image
// Each cell value: 0 (black) to 255 (white)
int[][] image = {
    {200, 150, 100},
    { 80, 120, 200},
    { 30,  60, 255}
};

// Invert the image (photo negative effect)
for (int r = 0; r < image.length; r++) {
    for (int c = 0; c < image[r].length; c++) {
        image[r][c] = 255 - image[r][c];
    }
}

// Print inverted image
for (int[] row : image) {
    for (int pixel : row) {
        System.out.printf("%4d", pixel);
    }
    System.out.println();
}
```

6.5 Search Engine: Word Index Builder

Search engines build inverted indexes to find documents quickly. Arrays and Strings are at the core of this process.

```
public class WordIndex {
    public static void buildIndex(String[] documents) {
        Map<String, List<Integer>> index = new HashMap<>();

        for (int i = 0; i < documents.length; i++) {
            String[] words = documents[i].toLowerCase().split("\\s+");
            for (String word : words) {
                index.computeIfAbsent(word, k -> new ArrayList<>()).add(i);
            }
        }

        // Search for a word
        String query = "java";
        System.out.println("'" + query + "' found in docs: " + index.get(query));
    }
}
```

Section 7: Conclusion

Summary and Next Steps

7.1 Key Concepts — Arrays

Concept	Summary
Definition	Fixed-size, ordered, same-type container stored contiguously
Index	Zero-based; first = <code>arr[0]</code> , last = <code>arr[arr.length-1]</code>
Types	1D, 2D, and multidimensional arrays
Initialization	Via literals <code>{}</code> , <code>new</code> keyword, or runtime input
Loops	<code>for</code> (with index), <code>while</code> , enhanced <code>for</code> (read-only)
Utilities	<code>java.util.Arrays</code> : <code>sort</code> , <code>search</code> , <code>fill</code> , <code>copy</code> , <code>equals</code>
Pass by ref	Arrays pass reference — method changes affect original
Copy	<code>Arrays.copyOf()</code> , <code>System.arraycopy()</code> , <code>clone()</code>

7.2 Key Concepts — Strings

Concept	Summary
Immutability	Strings cannot be changed; operations return new objects
Creation	Literals (pool) vs <code>new</code> keyword (heap)
Comparison	Always use <code>.equals()</code> , never <code>==</code> for content comparison
Key Methods	<code>length</code> , <code>charAt</code> , <code>substring</code> , <code>split</code> , <code>replace</code> , <code>contains</code> , <code>trim</code>
Performance	Use <code>StringBuilder</code> for concatenation in loops
Formatting	<code>String.format()</code> and <code>printf()</code> for formatted output
Manipulation	<code>split()</code> , <code>join()</code> , <code>repeat()</code> , <code>strip()</code> , <code>toCharArray()</code>

7.3 Learning Roadmap — What to Study Next

- **Collections Framework** — `ArrayList`, `LinkedList`, `HashMap` (dynamic alternatives to arrays)
- **Java Streams API** — powerful functional operations on arrays and collections
- **Regular Expressions** — advanced String pattern matching with `Pattern/Matcher`
- **Recursion** — solve array/String problems recursively
- **Sorting Algorithms** — `QuickSort`, `MergeSort`, and when to use each

- **Data Structures** — Stacks, Queues, Trees built from arrays
- **Algorithm Complexity** — Big-O notation to evaluate your solutions

■ Final Advice for Students

- Practice writing code daily — even 20 minutes makes a huge difference.
- Solve problems on LeetCode, HackerRank, or CodeChef starting with Easy level.
- Don't just read solutions — type them out and understand each line.
- Review the common mistakes section before every interview.
- Build small projects that use arrays and strings (e.g., a contact book, grade tracker).

■ **Happy Coding!**